

JavaScript Array Method - sort()

The `sort()` method is one of the most powerful array methods in JavaScript. It is used to **sort the elements of an array**.

By default, `sort()` arranges elements in **ascending alphabetical (Unicode) order**.

It **modifies the original array**.

Table of Contents

- What is sort()?
- Why Use sort()?
- Syntax
- Parameters
- Return Value
- How sort() Works
- Basic Examples
- Sorting Numbers
- Sorting Strings
- Custom Sorting
- Real Project Examples
- Best Practices
- Common Mistakes
- Interview Questions
- Summary Table

What is sort()?

Definition

The `sort()` method sorts the elements of an array.

By default:

- Strings are sorted alphabetically.

- Numbers are sorted as strings (Unicode order).

It modifies the **original array**.

Why Use sort()?

We use `sort()` when we need to:

- Sort numbers
- Sort names
- Display products alphabetically
- Sort prices
- Display rankings
- Arrange data before showing it to users

Syntax

```
array.sort();  
  
array.sort(compareFunction);
```

Parameters

Parameter	Description
compareFunction	Optional function used for custom sorting

Return Value

Returns the **sorted array**.

The original array is modified.

How sort() Works

Before

```
["Orange", "Apple", "Banana"]
```

↓

```
sort()
```

↓

After

```
["Apple", "Banana", "Orange"]
```

Example 1 (Basic String Sorting)

```
const fruits = [  
  "Orange",  
  "Apple",  
  "Banana"  
];  
  
fruits.sort();  
  
console.log(fruits);
```

Output

```
["Apple", "Banana", "Orange"]
```

Explanation

Strings are sorted alphabetically.

Example 2 (Numbers Without Compare Function)

```
const numbers = [  
  100,  
  20,  
  5,  
  40  
];  
  
numbers.sort();  
  
console.log(numbers);
```

Output

```
[100,20,40,5]
```

Explanation

JavaScript compares numbers as **strings** by default.

```
100  
  
20  
  
40  
  
5
```

are sorted alphabetically, not numerically.

Example 3 (Ascending Numbers)

```
const numbers = [  
  100,  
  20,  
  5,  
  40  
];  
  
numbers.sort(function(a,b){  
  return a-b;  
});  
  
console.log(numbers);
```

Output

```
[5,20,40,100]
```

Explanation

If

```
a-b
```

is negative,

a comes before **b** .

Example 4 (Descending Numbers)

```
const numbers = [  
  100,  
  20,  
  5,  
  40  
];  
  
numbers.sort(function(a,b){  
  return b-a;  
});  
  
console.log(numbers);
```

Output

```
[100,40,20,5]
```

Example 5 (Case Sensitive)

```
const names = [  
  "John",  
  "apple",  
  "Banana"  
];
```

```
names.sort();  
  
console.log(names);
```

Output

```
["Banana","John","apple"]
```

Uppercase letters come before lowercase letters.

Example 6 (Copy Before Sorting)

```
const numbers = [  
  30,  
  10,  
  20  
];  
  
const sorted = numbers.slice().sort(function(a,b){  
  return a-b;  
});  
  
console.log(sorted);  
  
console.log(numbers);
```

Output

```
[10,20,30]
```

```
[30,10,20]
```

The original array remains unchanged because `slice()` creates a copy.

Example 7 (Sort Objects)

```
const students = [  
  
  {  
  
    name:"Rahim",  
  
    marks:80  
  
  },  
  
  {  
  
    name:"Karim",  
  
    marks:95  
  
  },  
  
  {  
  
    name:"Rokon",  
  
    marks:70  
  
  }  
  
];  
  
students.sort(function(a,b){  
  
  return a.marks-b.marks;  
  
});  
  
console.log(students);
```

Output


```
[
  { name:"Rokon", marks:70 },
  { name:"Rahim", marks:80 },
  { name:"Karim", marks:95 }
]
```

Example 8 (Sort by String Length)

```
const words = [

  "Java",

  "HTML",

  "JavaScript",

  "CSS"

];

words.sort(function(a,b){

  return a.length-b.length;

});

console.log(words);
```

Output

```
[
  "CSS",
  "Java",
  "HTML",
  "JavaScript"
]
```

Real Project Example 1

Product Price

```
const prices = [  
  500,  
  200,  
  800,  
  300  
];  
  
prices.sort(function(a,b){  
  return a-b;  
});  
  
console.log(prices);
```

Output

```
[200,300,500,800]
```

Real Project Example 2

Student Marks

```
const marks = [  
  75,
```

```
    92,  
  
    60,  
  
    88  
  
];  
  
marks.sort(function(a,b){  
  
    return b-a;  
  
});  
  
console.log(marks);
```

Output

```
[92,88,75,60]
```

Real Project Example 3

Product Names

```
const products = [  
  
    "Mouse",  
  
    "Keyboard",  
  
    "Laptop"  
  
];  
  
products.sort();  
  
console.log(products);
```

Output

```
[  
  "Keyboard",  
  "Laptop",  
  "Mouse"  
]
```

Real Project Example 4

Leaderboard

```
const scores = [  
  
  900,  
  
  1200,  
  
  700  
];  
  
scores.sort(function(a,b){  
  
  return b-a;  
  
});  
  
console.log(scores);
```

Output

```
[1200,900,700]
```

Real Project Example 5

Employee Salary

```
const salaries = [  
  45000,  
  30000,  
  60000  
];  
  
salaries.sort(function(a,b){  
  return a-b;  
});  
  
console.log(salaries);
```

Output

```
[30000,45000,60000]
```

Best Practices

Always use a compare function when sorting numbers.

```
numbers.sort((a,b)=>a-b);
```

Use `slice()` before `sort()` if you don't want to modify the original array.

```
const sorted = array.slice().sort();
```

Use meaningful variable names.

```
sortedProducts
```

```
sortedMarks
```

```
sortedStudents
```

Common Mistakes

Mistake 1

Sorting numbers without a compare function.

Wrong

```
[100,20,5].sort();
```

Output

```
[100,20,5]
```

Correct

```
[100,20,5].sort((a,b)=>a-b);
```

Output

```
[5,20,100]
```

Mistake 2

Thinking `sort()` creates a new array.

```
const arr = [3,1,2];  
  
arr.sort();  
  
console.log(arr);
```

Output

```
[1,2,3]
```

The original array changes.

Mistake 3

Ignoring uppercase and lowercase letters.

```
["apple","Banana"].sort();
```

Output

```
["Banana","apple"]
```

Mistake 4

Using `reverse()` instead of descending sort.

Wrong

```
numbers.reverse();
```

Correct

```
numbers.sort((a,b)=>b-a);
```

Interview Questions

What does `sort()` do?

It sorts the elements of an array.

Does `sort()` modify the original array?

Yes.

What does `sort()` return?

The sorted array.

Why should we use a compare function for numbers?

Because JavaScript sorts numbers as strings by default.

How do you sort numbers in ascending order?

```
numbers.sort((a,b)=>a-b);
```

How do you sort numbers in descending order?


```
numbers.sort((a,b)=>b-a);
```

How can you sort without modifying the original array?

```
const sorted = array.slice().sort((a,b)=>a-b);
```

Summary Table

| Feature | Description | ||-| | Method | `sort()` | | Purpose | Sort array elements | | Parameters | compareFunction (optional) | | Returns | Sorted array | | Changes Original Array | Yes | | Default Sort | Unicode (Alphabetical) | | Common Uses | Products, Prices, Rankings, Leaderboards |

Quick Comparison

Method	Purpose	Returns	Changes Original Array
<code>sort()</code>	Sort elements	Sorted Array	Yes
<code>reverse()</code>	Reverse order	Reversed Array	Yes
<code>slice()</code>	Copy array	New Array	No

Final Notes

The `sort()` method is one of the most frequently used array methods in JavaScript. It is essential for organizing data before displaying it to users.

Common real-world uses include:

- Product sorting (Price Low → High)
- Student ranking
- Leaderboards
- Alphabetical lists
- Employee salary sorting

- Search results
- E-commerce product filtering

Remember: Always use a **compare function** when sorting numbers. Otherwise, JavaScript sorts them as strings, which can produce unexpected results.